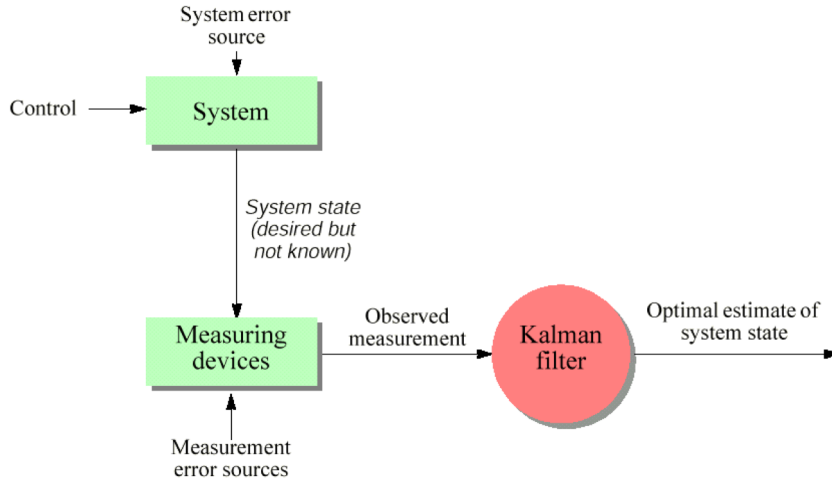


Map-based navigation

Kalman filter localization

Kalman filter localization



Dead Reckoning

- *Dead reckoning* is the estimation of a robot's pose based on its estimated speed, direction, and time of travel with respect to a previous estimate
- An **odometer** is a sensor that measures distance travelled and sometimes also change in heading direction
 - For wheeled robots this can be determined by measuring the angular rotation of the wheels
 - For other robots, e.g., aerial and underwater, one can use **visual odometry**

Model of the vehicle

When the vehicle move forward in the x -direction by δ_d , and rotate by δ_θ , the new position can be computed as

$$\mathbf{x}(k+1) = \mathbf{x}(k) + \begin{bmatrix} \delta_d \cos \theta(k) \\ \delta_d \sin \theta(k) \\ \delta_\theta \end{bmatrix}$$

Model of the vehicle

When the vehicle move forward in the x -direction by δ_d , and rotate by δ_θ , the new position can be computed as

$$\mathbf{x}(k+1) = \mathbf{x}(k) + \begin{bmatrix} \delta_d \cos \theta(k) \\ \delta_d \sin \theta(k) \\ \delta_\theta \end{bmatrix}$$

The measured output of a real odometer is the odometry $\delta = (\delta_d, \delta_\theta)$ plus the output of the random number generator $\mathbf{v} = (v_d, v_\theta)$, resulting in

$$\mathbf{x}(k+1) = \mathbf{f}(\mathbf{x}(k), \delta(k), \mathbf{v}(k)) = \mathbf{x}(k) + \begin{bmatrix} (\delta_d + v_d) \cos \theta(k) \\ (\delta_d + v_d) \sin \theta(k) \\ \delta_\theta + v_\theta \end{bmatrix}$$

Uncertainty modeling

- In the absence of any information to the contrary, we **model the odometry noise** as

$$\mathbf{v} = [v_d, v_\theta]^\top \sim \mathcal{N}(\mathbf{0}, \mathbf{V})$$

i.e., a zero-mean multivariate Gaussian process with covariance matrix

$$\mathbf{V} = \begin{bmatrix} \sigma_d^2 & 0 \\ 0 & \sigma_\theta^2 \end{bmatrix}$$

Estimating pose

The Kalman filter is a **recursive algorithm** for the estimate of the pose. It consists of two steps:

- **Prediction** of the next pose
- **Correction** of the pose

Estimating the pose – Prediction

The **prediction equations** are

$$\begin{aligned}\hat{\mathbf{x}}(k+1|k) &= \mathbf{f}(\hat{\mathbf{x}}(k|k), \mathbf{u}(k)) \\ \hat{\mathbf{P}}(k+1|k) &= \mathbf{F}_x \hat{\mathbf{P}}(k|k) \mathbf{F}_x^\top + \mathbf{F}_v \hat{\mathbf{V}} \mathbf{F}_v^\top\end{aligned}$$

- $\hat{\mathbf{x}}(k+1|k)$ is the **prediction** of the state at time $k+1$ given the information available at time k
- $\hat{\mathbf{x}}(k|k)$ is the best **estimate** of the state at time k
- $\mathbf{u}(k)$ is the input of the process at time k
- $\hat{\mathbf{P}} \in \mathbb{R}^{3 \times 3}$ is a covariance matrix representing uncertainty in the estimated vehicle configuration
- $\hat{\mathbf{V}}$ is the estimate of the covariance of the odometry noise

Estimating the pose – Prediction

The **prediction equations** are

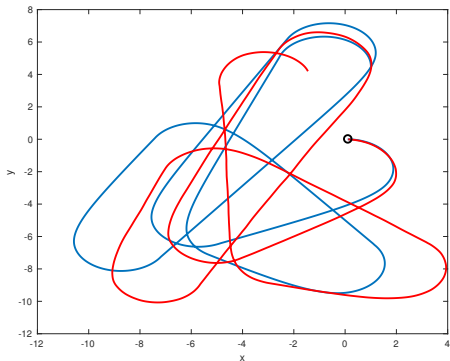
$$\begin{aligned}\hat{\mathbf{x}}(k+1|k) &= \mathbf{f}(\hat{\mathbf{x}}(k|k), \mathbf{u}(k)) \\ \hat{\mathbf{P}}(k+1|k) &= \mathbf{F}_x \hat{\mathbf{P}}(k|k) \mathbf{F}_x^\top + \mathbf{F}_v \hat{\mathbf{V}} \mathbf{F}_v^\top\end{aligned}$$

- while $\mathbf{F}_x$ and $\mathbf{F}_v$ are the Jacobians:

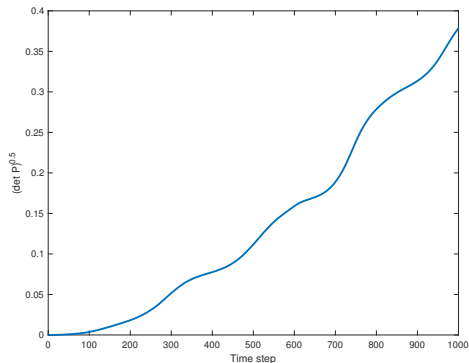
$$\begin{aligned}\mathbf{F}_x &= \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\mathbf{v}=\mathbf{0}} = \begin{bmatrix} 1 & 0 & -\delta_d \sin \theta \\ 0 & 1 & \delta_d \cos \theta \\ 0 & 0 & 1 \end{bmatrix} \\ \mathbf{F}_v &= \left. \frac{\partial \mathbf{f}}{\partial \mathbf{v}} \right|_{\mathbf{v}=\mathbf{0}} = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ 0 & 1 \end{bmatrix}\end{aligned}$$

Only the prediction step is not sufficient...

True path (blue) vs Estimated path (red)



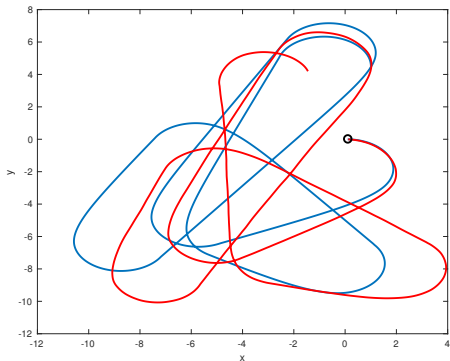
Entity of the uncertainty over time



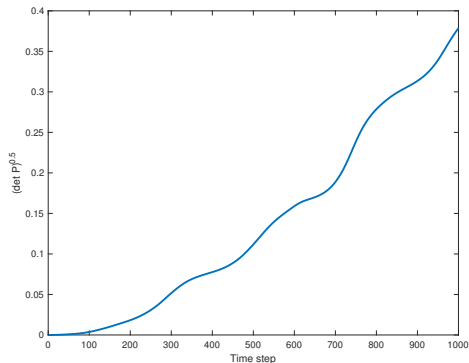
Check `EKF_Example.m`

Only the prediction step is not sufficient...

True path (blue) vs Estimated path (red)



Entity of the uncertainty over time



We need to **correct** the prediction with some measurements!

Check `EKF_Example.m`

Estimating the pose – Correction (1)

- For improving the estimate, we can rely on **Landmarks** whose **positions are known**

Estimating the pose – Correction (1)

- For improving the estimate, we can rely on **Landmarks** whose **positions are known**
- The robot is equipped with a sensor that provides observations of the landmarks **with respect to the robot** as described by

$$\mathbf{z} = \mathbf{h}(\mathbf{x}, \mathbf{p}_i)$$

where $\mathbf{p}_i = [x_i, y_i]^\top$ is the *known* location of the i -th landmark

Estimating the pose – Correction (1)

- For improving the estimate, we can rely on **Landmarks** whose **positions are known**
- The robot is equipped with a sensor that provides observations of the landmarks **with respect to the robot** as described by

$$\mathbf{z} = \mathbf{h}(\mathbf{x}, \mathbf{p}_i)$$

where $\mathbf{p}_i = [x_i, y_i]^\top$ is the *known* location of the i -th landmark

- A common type of sensor measures the range and bearing angle to a landmark in the environment, e.g., radar, scanning-laser rangefinder, Lidar, ...

Estimating the pose – Correction (1)

- For improving the estimate, we can rely on **Landmarks** whose **positions are known**
- The robot is equipped with a sensor that provides observations of the landmarks **with respect to the robot** as described by

$$\mathbf{z} = \mathbf{h}(\mathbf{x}, \mathbf{p}_i)$$

where $\mathbf{p}_i = [x_i, y_i]^\top$ is the *known* location of the i -th landmark

- A common type of sensor measures the range and bearing angle to a landmark in the environment, e.g., radar, scanning-laser rangefinder, Lidar, ...
- The **measurement model** is therefore

$$\mathbf{z} = \mathbf{h}(\mathbf{x}, \mathbf{p}_i) = \begin{bmatrix} \sqrt{(y_i - y)^2 + (x_i - x)^2} \\ \text{atan2} \left(\frac{y_i - y}{x_i - x} \right) - \theta \end{bmatrix} + \begin{bmatrix} w_r \\ w_\beta \end{bmatrix}$$

where $\mathbf{z} = [r, \beta]^\top$, r : range, β : bearing angle, and $\mathbf{w} = [w_r, w_\beta]^\top \sim \mathcal{N}(\mathbf{0}, \mathbf{W})$,
 $\mathbf{W} = \text{diag}(\sigma_r^2, \sigma_\beta^2)$

Estimating the pose – Correction (2)

- We define the **innovation**

$$\mathbf{v} = \mathbf{z}(k+1) - \mathbf{h}(\hat{\mathbf{x}}(k+1|k), \mathbf{p}_i)$$

Estimating the pose – Correction (2)

- We define the **innovation**

$$\mathbf{v} = \mathbf{z}(k+1) - \mathbf{h}(\hat{\mathbf{x}}(k+1|k), \mathbf{p}_i)$$

- The **correction equations** are

$$\hat{\mathbf{x}}(k+1|k+1) = \hat{\mathbf{x}}(k+1|k) + \mathbf{K}\mathbf{v}$$

$$\hat{\mathbf{P}}(k+1|k+1) = \hat{\mathbf{P}}(k+1|k) - \mathbf{K}\mathbf{H}_x\hat{\mathbf{P}}(k+1|k)$$

Estimating the pose – Correction (2)

- We define the **innovation**

$$\mathbf{v} = \mathbf{z}(k+1) - \mathbf{h}(\hat{\mathbf{x}}(k+1|k), \mathbf{p}_i)$$

- The **correction equations** are

$$\hat{\mathbf{x}}(k+1|k+1) = \hat{\mathbf{x}}(k+1|k) + \mathbf{K}\mathbf{v}$$

$$\hat{\mathbf{P}}(k+1|k+1) = \hat{\mathbf{P}}(k+1|k) - \mathbf{K}\mathbf{H}_x\hat{\mathbf{P}}(k+1|k)$$

- and the **Kalman gain update equations** are

$$\mathbf{K} = \hat{\mathbf{P}}(k+1|k)\mathbf{H}_x^\top \mathbf{S}^{-1}$$

$$\mathbf{S} = \mathbf{H}_x\hat{\mathbf{P}}(k+1|k)\mathbf{H}_x^\top + \mathbf{H}_w\hat{\mathbf{W}}\mathbf{H}_w^\top$$

Estimating the pose – Correction (2)

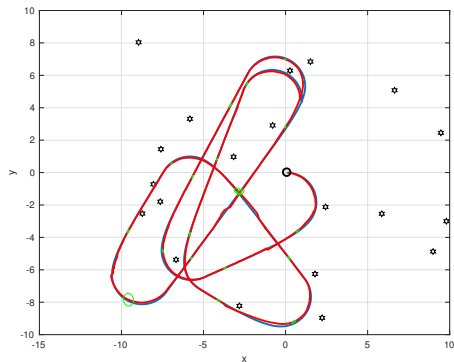
- In the Kalman gain update equations, the quantities

$$\mathbf{H}_x = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{w}=\mathbf{0}} = \begin{bmatrix} -\frac{x_i - x}{r} & -\frac{y_i - y}{r} & 0 \\ \frac{y_i - y}{r^2} & -\frac{x_i - x}{r^2} & -1 \end{bmatrix}$$
$$\mathbf{H}_w = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{w}} \right|_{\mathbf{w}=\mathbf{0}} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

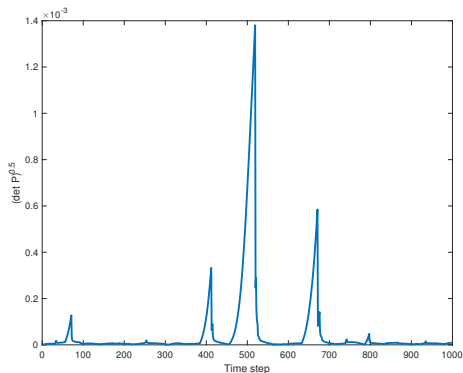
with $r = \sqrt{(y_i - y)^2 + (x_i - x)^2}$

Effect of Prediction + Correction

True path (blue) vs Estimated path (red)



Entity of the uncertainty over time

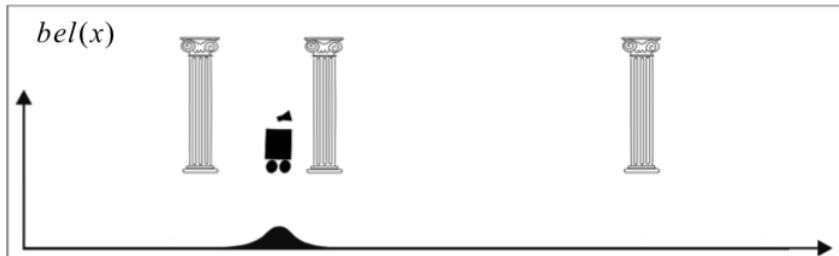


Check `EKF_correctionPrediction.m`

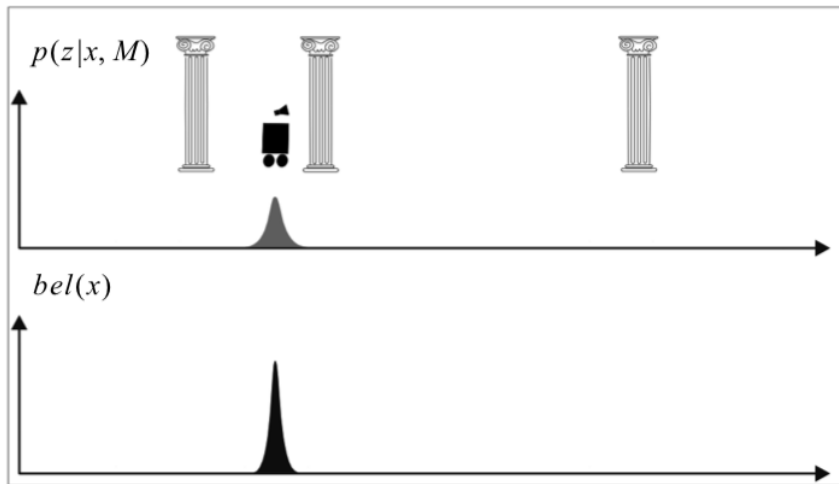
Example on the 1D robot



Example on the 1D robot



Example on the 1D robot



Map-based navigation

Particle filter

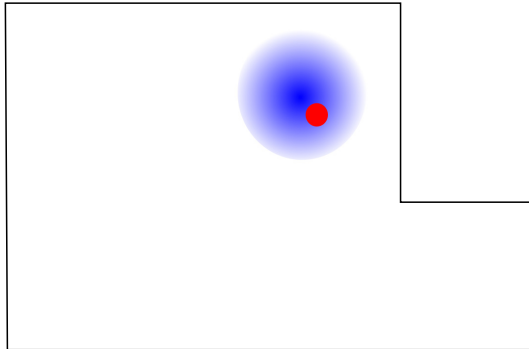
Particle filter

- Kalman Filter (and its variants) are trying to **estimate the position with a Gaussian distribution**, where the estimated position $\hat{\mathbf{x}}$ is the mean, with its associated covariance $\hat{\mathbf{P}}$.
- Gaussian filters are struggling when
 - The measurement and motion model are **highly nonlinear**
 - The posterior distribution is **significantly non Gaussian**

Particle filter

- In the case of the EKF, the initial position $\mathbf{x}(0)$ is known with uncertainty $\mathbf{P}(0)$.
- But..
 - what if **we do not know the initial position**?
 - or the initial position is given from a **non-Gaussian distribution**?

2D Gaussian distribution



- For these problems, we can use the **Particle filter** (a.k.a Monte Carlo localisation)

Particle filter

- The particle filter does **not** approximate the position (i.e the **posterior**, $p(x)$) in a typical distribution
- Can deal with highly nonlinear models
- The particle filter estimates the **posterior using a set of weighted samples**, i.e **particles**

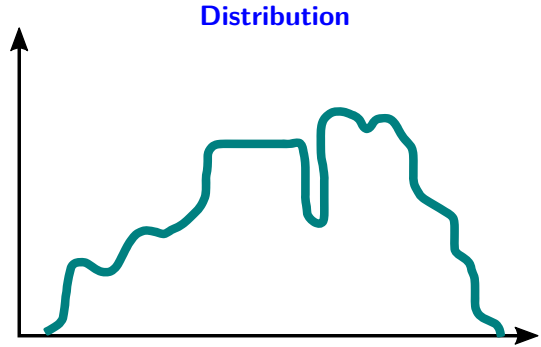
$$\hat{p}(\mathbf{x}) = \sum_{i=1}^N w_i \delta_{\mathbf{x}_i}(\mathbf{x})$$

where $\delta(\mathbf{x})$ is the **Dirac delta** function, i is the current particle, with estimated pose $\mathbf{x}_i$, and N is the total number of particles

- Drawbacks
 - **Computational cost**, we need to keep track of all particles instead of just a mean and a covariance matrix as in the case of the EKF.

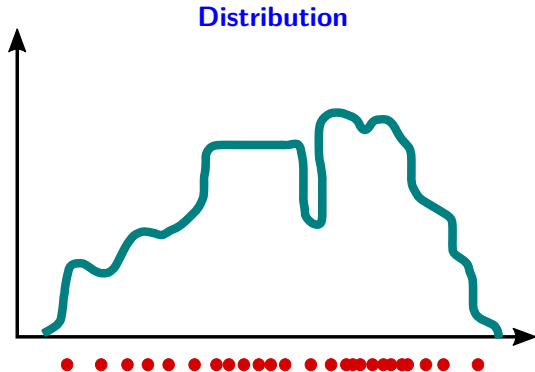
1D case

- Obviously not Gaussian
- So, how can we represent this distribution with samples?



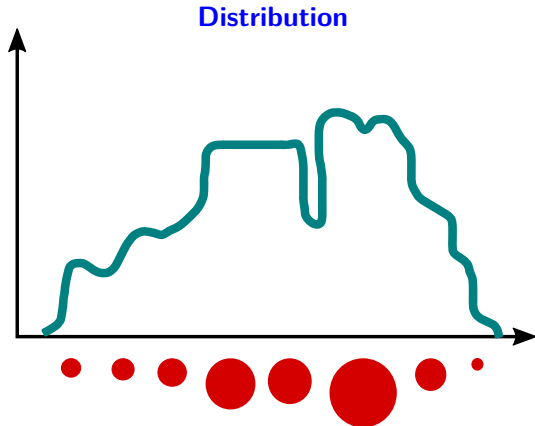
1D case

- Obviously not Gaussian
- So, how can we represent this distribution with samples?
 1. Number of samples in region corresponds to the probability



1D case

- Obviously not Gaussian
- So, how can we represent this distribution with samples?
 1. Number of samples in region corresponds to the probability
 2. **Weighted samples**
- The **more samples**, the **better approximation**



Particle filter

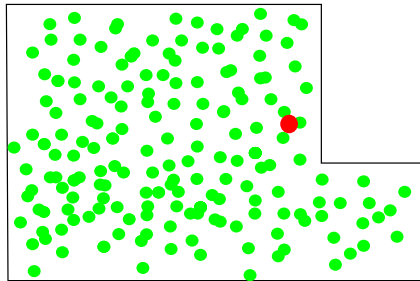
- The particle filter can be divided into five steps
 1. **Initialise** the particle set
 2. **Predict** where the particles will be using a **motion model**
 3. **Update the weights** of the particles
 4. Normalise the weights and compute the **estimated position**
 5. **Re-sample** the best particles
- Repeat step 2-5

1. Initialise particle set

- How can we use the particle filter when **we do not know the initial position?**

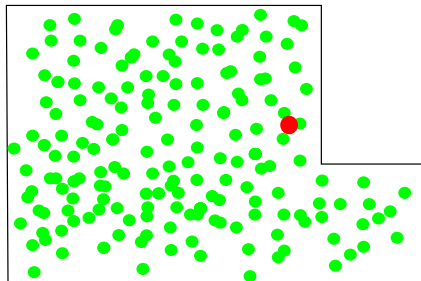
1. Initialise particle set

- How can we use the particle filter when **we do not know the initial position?**
- **Randomly distribute** particles around the **whole map**



1. Initialise particle set

- How can we use the particle filter when **we do not know the initial position?**
- **Randomly distribute** particles around the **whole map**
- Each particle's weight is initialised as $w_i = 1/N$, where N is the number of particles



2. Prediction

Propagate all particles according to a motion model, for example:

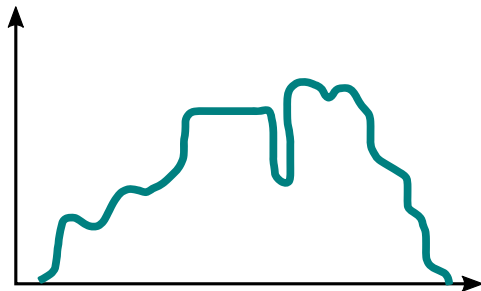
$$\hat{\mathbf{x}}(k+1) = \mathbf{f}(\mathbf{x}(k), \delta(k), \mathbf{r}_i(k)) = \mathbf{x}(k) + \begin{bmatrix} (\delta_d + r_{d_i}) \cos \theta(k) \\ (\delta_d + r_{d_i}) \sin \theta(k) \\ \delta_\theta + r_{\theta_i} \end{bmatrix}$$

Note

- $\mathbf{r}_i(k)$ will be a **new random number for each particle and each time instant**
- $\mathbf{r}_i(k)$ can be drawn from **any distribution**
- r_{d_i} and r_{θ_i} can use different distributions

3. Weight update

- Problem - we want to sample particles from a distribution, $h(x)$, we do not know how to sample from

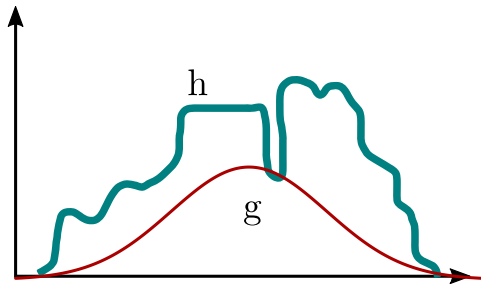


3. Weight update

- Problem - we want to sample particles from a distribution, $h(x)$, we do not know how to sample from
- Idea - use a distribution that we do know how to sample from, for example a Gaussian distribution $g(x)$, and **account for the difference with a weight**
- This is usually referred to as **importance sampling**
- The weight w of particle i is:

$$w_i = \frac{h(x)}{g(x)}$$

- Important that $h(x) \geq 0 \rightarrow g(x) \geq 0$



3. Weight update

- The particle filter **does not** use the measurement from external sensors to correct the predicted estimate.

3. Weight update

- The particle filter **does not** use the measurement from external sensors to correct the predicted estimate.
- The **measurements** are used to compute the **weight of each particle**

3. Weight update

- The particle filter **does not** use the measurement from external sensors to correct the predicted estimate.
- The **measurements** are used to compute the **weight of each particle**
 - "Given my predicted position, how likely is it that I measure this range?"

$$w_i(k) = \frac{\text{target}}{\text{proposal}} = \frac{h(x)}{g(x)} \propto p(\mathbf{z}(k) | \mathbf{x}_i(k), \mathbf{m})$$

where

- $w_i(k)$ is the weight of particle i , at time k
 - $\mathbf{z}(k)$ is the measurement of our external sensor at time k
 - $\mathbf{x}_i(k)$ is the predicted pose at time k for particle i
 - $\mathbf{m}$ is the information we have about the map, for example, the localisation of landmarks
- The particles with a more likely predicted position gets a higher weight

3. Weight update

- The particle filter **does not** use the measurement from external sensors to correct the predicted estimate.
- The **measurements** are used to compute the **weight of each particle**
 - "Given my predicted position, how likely is it that I measure this range?"

$$w_i(k) = \frac{\text{target}}{\text{proposal}} = \frac{h(x)}{g(x)} \propto p(\mathbf{z}(k) | \mathbf{x}_i(k), \mathbf{m})$$

where

- $w_i(k)$ is the weight of particle i , at time k
 - $\mathbf{z}(k)$ is the measurement of our external sensor at time k
 - $\mathbf{x}_i(k)$ is the predicted pose at time k for particle i
 - $\mathbf{m}$ is the information we have about the map, for example, the localisation of landmarks
- The particles with a more likely predicted position gets a higher weight
 - The question is

How do we compute these weights?

3. Weight update

- We can use our **measurements** $\mathbf{z}(k)$, and our **measurement model**

$$\mathbf{z} = \mathbf{h}(\mathbf{x}_i, \mathbf{p}_j) = \begin{bmatrix} \sqrt{(y_j - y_i)^2 + (x_j - x_i)^2} \\ \text{atan2} \left(\frac{y_j - y_i}{x_j - x_i} \right) - \theta \end{bmatrix} + \begin{bmatrix} w_r \\ w_\beta \end{bmatrix}$$

- We need to compute this for **all particles at each time instant**

3. Weight update

- If we assume **Gaussian noise for our measurements**, we can compute the likelihood for our sample using a Gaussian **probability density function** (other PDFs might be used if we assume the noise to have a different distribution).

$$w_i = \prod_j \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{h(\mathbf{x}_i, p_j) - z_j}{\sigma}\right)^2\right)$$

where

- w_i is the weight of particle i
- L is the number of landmarks
- σ is the standard deviation of the measurement noise
- $h(\mathbf{x}_i, p_j)$ is the **estimated measurement** from particle $\mathbf{x}_i$ to the **known position** of landmark p_j
- z_j is the sensor measurement from the robots position to the landmark j

4. Estimate position

- Before we **estimate the position**, we need to **normalise** the weights

$$w_i = \frac{w_i}{\sum_{i=1}^N}$$

4. Estimate position

- Before we **estimate the position**, we need to **normalise** the weights

$$w_i = \frac{w_i}{\sum_{i=1}^N}$$

- For the x and y position it is straightforward to compute the weighted estimate

$$\hat{x} = \sum_{i=1}^N x_i w_i, \quad \hat{y} = \sum_{i=1}^N y_i w_i,$$

4. Estimate position

- Before we **estimate the position**, we need to **normalise** the weights

$$w_i = \frac{w_i}{\sum_{i=1}^N}$$

- For the x and y position it is straightforward to compute the weighted estimate

$$\hat{x} = \sum_{i=1}^N x_i w_i, \quad \hat{y} = \sum_{i=1}^N y_i w_i,$$

- We need to be **more careful** when we deal with **angles**...

4. Estimate position

- What is average of 359° and 1° ?

4. Estimate position

- What is average of 359° and 1° ?
- We can turn to our old friend atan2 !
- First, we convert our angles to **points on the unit circle**, i.e $p_i = (\cos(\theta_i), \sin(\theta_i))$
- Then compute the weighted average of these points

4. Estimate position

- What is average of 359° and 1° ?
- We can turn to our old friend atan2!
- First, we convert our angles to **points on the unit circle**, i.e $p_i = (\cos(\theta_i), \sin(\theta_i))$
- Then compute the weighted average of these points
- Compute the weighted mean angle using atan2

$$\hat{\theta} = \text{atan2} \left(\sum_{i=1}^N w_i \sin(\theta_i), \sum_{i=1}^N w_i \cos(\theta_i) \right)$$

Re-sampling

A **problem** with the particle filter is the fact that as time goes we would eventually have **one particle** with a weight close to 1, and all other particles would have a weight close to 0.

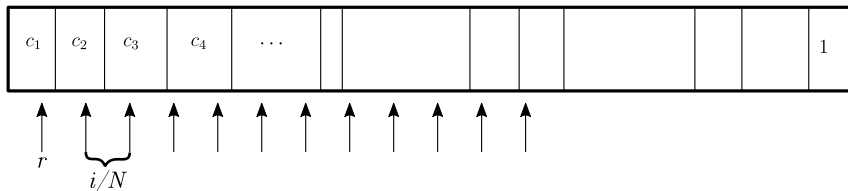
- We would still have to update the particles with low weight
- What if this promising particle is wrong?

Introducing the **re-sample step**

- Generate N new samples, but this time, not randomly over the entire state space
- Draw samples from the particle set, the **probability** to draw particle i is **proportional** to w_i
- Create samples close to the most likely samples
- Numerous of ways to perform the re-sampling step
- We will use the **low variance re-sampling** algorithm

Re-sampling

- Draw **one** uniform random number between 0 and $1/N$, i.e $r \sim \mathcal{U}(0, N^{-1})$
- Compute an array of the cumulative weights,
 $c_1 = w_1, c_2 = w_1 + w_2, \dots, c_N = \sum_{i=1}^N w_i$
- The particle with the weight w at location in the array corresponding to $u_1 = r$ will be chosen as the first particle to be re-sampled
- Next, increment with a step size $r + (i - 1)/N$, i.e $u_i = r + (i - 1)/N$, for $i = 1$ to N .



Algorithm 1 Low variance re-sampling ($\mathbf{x}, \mathbf{w}$)

```
 $\mathbf{x}_{k+1} = \emptyset$   
 $r = \text{rand}(0; N^{-1}), c = w_1, j = 1$   
for  $i = 1$  to  $N$  do do  
     $U = r + (i - 1) \cdot N^{-1}$   
    while  $U > c$  do  
         $j = j + 1$   
         $c = c + w_j$   
    end while  
    add  $x_j$  to  $\mathbf{x}_{k+1}$   
end for  
Return  $\mathbf{x}_{k+1}$ 
```

Re-sampling

Go back to the prediction step

Check `particleFilter_Example.m`

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping

Creating a map

Creating a map (1)

- We now assume that we know perfectly the position of the robot
- We want to estimate the coordinates of the **landmarks**

Creating a map (1)

- We now assume that we know perfectly the position of the robot
- We want to estimate the coordinates of the **landmarks**
- The state vector is

$$\hat{\mathbf{x}} = [x_1 \quad y_1 \quad x_2 \quad y_2 \quad \cdots \quad x_M \quad y_M]^T \in \mathbb{R}^{2M \times 1}$$

Creating a map (1)

- We now assume that we know perfectly the position of the robot
- We want to estimate the coordinates of the **landmarks**
- The state vector is

$$\hat{\mathbf{x}} = [x_1 \quad y_1 \quad x_2 \quad y_2 \quad \cdots \quad x_M \quad y_M]^T \in \mathbb{R}^{2M \times 1}$$

- and the estimated covariance matrix $\hat{\mathbf{P}} \in \mathbb{R}^{2M \times 2M}$

Creating a map (1)

- We now assume that we know perfectly the position of the robot
- We want to estimate the coordinates of the **landmarks**
- The state vector is

$$\hat{\mathbf{x}} = [x_1 \quad y_1 \quad x_2 \quad y_2 \quad \cdots \quad x_M \quad y_M]^T \in \mathbb{R}^{2M \times 1}$$

- and the estimated covariance matrix $\hat{\mathbf{P}} \in \mathbb{R}^{2M \times 2M}$
- We assume the landmarks to be **stationary**, i.e.

$$\hat{\mathbf{x}}(k+1|k) = \hat{\mathbf{x}}(k|k)$$

$$\hat{\mathbf{P}}(k+1|k) = \hat{\mathbf{P}}(k|k)$$

Creating a map (2)

- We introduce the function $\mathbf{g}(\cdot)$ which is the inverse of $\mathbf{h}(\cdot)$ and gives the coordinates of the observed landmark based on the known vehicle pose $\mathbf{x}_v = (x_v, y_v, \theta_v)$, i.e.

$$\mathbf{g}(\mathbf{x}_v, \mathbf{z}) = \begin{bmatrix} x_v + r \cos(\theta_v + \beta) \\ y_v + r \sin(\theta_v + \beta) \end{bmatrix}$$

Creating a map (2)

- We introduce the function $\mathbf{g}(\cdot)$ which is the inverse of $\mathbf{h}(\cdot)$ and gives the coordinates of the observed landmark based on the known vehicle pose $\mathbf{x}_v = (x_v, y_v, \theta_v)$, i.e.

$$\mathbf{g}(\mathbf{x}_v, \mathbf{z}) = \begin{bmatrix} x_v + r \cos(\theta_v + \beta) \\ y_v + r \sin(\theta_v + \beta) \end{bmatrix}$$

- Note that $\hat{\mathbf{x}}$ has a **variable length**, and we need to **extend** it and the covariance matrix whenever we encounter a landmark we have not previously seen.

Creating a map (2)

- We introduce the function $\mathbf{g}(\cdot)$ which is the inverse of $\mathbf{h}(\cdot)$ and gives the coordinates of the observed landmark based on the known vehicle pose $\mathbf{x}_v = (x_v, y_v, \theta_v)$, i.e.

$$\mathbf{g}(\mathbf{x}_v, \mathbf{z}) = \begin{bmatrix} x_v + r \cos(\theta_v + \beta) \\ y_v + r \sin(\theta_v + \beta) \end{bmatrix}$$

- Note that $\hat{\mathbf{x}}$ has a **variable length**, and we need to **extend** it and the covariance matrix whenever we encounter a landmark we have not previously seen.
 - The state vector is extended by the function $\mathbf{y}(\cdot)$

$$\mathbf{x}'(k) = \mathbf{y}(\mathbf{x}(k), \mathbf{z}(k), \mathbf{x}_v(k)) = \begin{bmatrix} \mathbf{x}(k) \\ \mathbf{g}(\mathbf{x}_v(k), \mathbf{z}(k)) \end{bmatrix}$$

Creating a map (2)

- We introduce the function $\mathbf{g}(\cdot)$ which is the inverse of $\mathbf{h}(\cdot)$ and gives the coordinates of the observed landmark based on the known vehicle pose $\mathbf{x}_v = (x_v, y_v, \theta_v)$, i.e.

$$\mathbf{g}(\mathbf{x}_v, \mathbf{z}) = \begin{bmatrix} x_v + r \cos(\theta_v + \beta) \\ y_v + r \sin(\theta_v + \beta) \end{bmatrix}$$

- Note that $\hat{\mathbf{x}}$ has a **variable length**, and we need to **extend** it and the covariance matrix whenever we encounter a landmark we have not previously seen.

- The state vector is extended by the function $\mathbf{y}(\cdot)$

$$\mathbf{x}'(k) = \mathbf{y}(\mathbf{x}(k), \mathbf{z}(k), \mathbf{x}_v(k)) = \begin{bmatrix} \mathbf{x}(k) \\ \mathbf{g}(\mathbf{x}_v(k), \mathbf{z}(k)) \end{bmatrix}$$

- The covariance matrix also needs to be extended as

$$\hat{\mathbf{P}}'(k) = \mathbf{Y}_z \begin{bmatrix} \hat{\mathbf{P}}(k) & 0 \\ 0 & \hat{\mathbf{W}} \end{bmatrix} \mathbf{Y}_z^\top, \quad \mathbf{Y}_z = \frac{\partial \mathbf{y}}{\partial \mathbf{z}} = \begin{bmatrix} \mathbf{I}_{n \times n} & 0_{n \times 2} \\ \mathbf{G}_x & 0_{2 \times n-3} & \mathbf{G}_z \end{bmatrix}$$

Creating a map (3)

- Where n is the dimension of $\hat{\mathbf{P}}(k)$

$$\mathbf{G}_x = \frac{\partial \mathbf{g}}{\partial \mathbf{x}} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

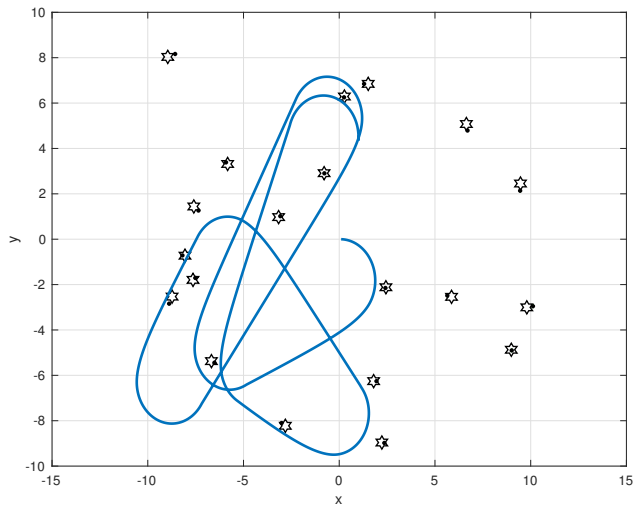
$$\mathbf{G}_z = \frac{\partial \mathbf{g}}{\partial \mathbf{z}} = \begin{bmatrix} \cos(\theta_v + \beta) & -r \sin(\theta_v + \beta) \\ \sin(\theta_v + \beta) & r \cos(\theta_v + \beta) \end{bmatrix}$$

- The Kalman filter equations for estimating the state are the same as before, for the estimate of the position of the landmarks $\hat{\mathbf{x}}$ with

$$\mathbf{H}_x = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\mathbf{w}=0} = [\mathbf{0} \quad \dots \quad \mathbf{H}_{\mathbf{p}_i} \quad \dots \quad \mathbf{0}] \in \mathbb{R}^{2 \times 2M}$$

$$\mathbf{H}_{\mathbf{p}_i} = \frac{\partial \mathbf{h}}{\partial \mathbf{p}_i} = \begin{bmatrix} \frac{x_i - x_v}{r} & \frac{y_i - y_v}{r} \\ -\frac{y_i - y_v}{r^2} & \frac{x_i - x_v}{r^2} \end{bmatrix}$$

Create a map



Check `createMap.m`